

RAIDAR 复现到“可对齐基线 + 服务提示词集成与路由”的程度

执行摘要

在你的开题主线“提示词集成 + 路由（Routing），并强调可控成本、优先用 deepseek-chat 等低成本重写器”的约束下，RAIDAR 复现不应追求把论文所有表格 1:1 复刻，而应做到两件事：第一，把 RAIDAR 的 **Invariance（单次重写）管线跑通**，复现其机制：LLM 重写人类文本通常改动更大、重写机器文本更“惯性”而改动更小，并且这种差异足以让一个轻量分类器学到有效边界；第二，**系统化复现提示词敏感性**（不同重写提示词导致检测性能显著波动、且没有单一最优提示词），从而为你后续的提示词池/集成/路由提供“必须做集成”的前置证据与实验底座。¹

本报告把复现范围收敛为你提出的 3 个主战场数据域：XSum（新闻）、PubMedQA（医学文本/问答语境）、WritingPrompts（创意写作），其余 Yelp/Code/ArXiv 仅作为可选扩展；重写器默认采用 deepseek-chat 控成本，同时用小规模对照证明“用 DeepSeek 替代 GPT-3.5 做重写时，机制与趋势仍成立”，避免评审质疑“没复现、只做变体”。²

复现范围分级与里程碑交付

你要复现到什么程度（与开题最匹配）

层级	你要做什么	为什么这最“值”	成本/风险控制点
必做：Baseline（支撑后续全部实验）	只复现 Invariance（单次重写） ：对输入只重写 1 次 → 提取 bag-of-words / n-gram overlap + Levenshtein ratio → LR / XGBoost 二分类	RAIDAR 核心机制在 Invariance 上最“便宜且关键”；论文也强调单个重写提示就能达到很高 F1，适合你先把地基打牢。 ³	重写结果必须缓存；固定抽样 ID；固定超参/随机种子
必做：Prompt sensitivity（前置证据）	设计 3-8 个重写提示词（流畅、同义改写、文体变换、压缩/扩展、句法重组等）逐一跑 Invariance，输出每提示词的指标	论文明确展示：不同重写提示词会显著影响检测性能，且 没有单一提示词在所有数据源上最强 ；这正是你做提示词集成/路由的直接动机。 ⁴	提示词池要版本化；加入长度控制与无长度控制的对照
建议做：DeepSeek vs GPT-3.5 小对照	每个数据集抽 100 human + 100 AI，分别用 gpt-3.5 与 deepseek-chat 做重写（仅 Invariance + LR）	论文主实验用 GPT-3.5-turbo 做重写；你换成 DeepSeek 后，需要一个“趋势一致性”证据，确保评审承认你是“复现机制 + 做低成本替代”。 ⁵	小样本即可；固定同一批 ID；只对比分布与趋势，不要求数值完全一致

层级	你要做什么	为什么这最“值”	成本/风险控制点
不建议做（当前阶段）	Equivariance/Uncertainty 全量复现；表 4/5 那种大规模网格	需要多次/多阶段重写，调用次数线性倍增甚至平方增长，且与你的创新点相关性较弱。论文也指出 GPT-3.5 代价高并讨论用更小重写模型的影响。 ⁶	可作为论文后期附录加分项，而非开题主线必需

里程碑拆解（按“可交付物”验收）

以下里程碑沿用你给出的结构，并补充“必须记录/必须缓存/必须对照”的复现工程规范。

Milestone 0：工程骨架搭好

目标：一条命令跑通 `load data → (generate AI data) → rewrite → feature → train → eval → export`，并保证可复跑不烧钱。

交付物（建议目录规范）：

- `configs/*.yaml`：数据集、采样规模、提示词池、重写模型（deepseek/gpt）、温度/长度约束、随机种子
- `python -m raider_repro.run --config configs/xsum_baseline.yaml`
- `outputs/{run_id}/`：
- `raw/`：抽样后的 human/ai 文本 JSONL
- `rewrite/`：按（数据集×提示词×重写器）缓存的 rewrite JSONL
- `features/`：Parquet/NPZ
- `metrics.json`、`confusion_matrix.json`、`plots/`（直方图/箱线图）

关键要求（踩坑高发点）：

- **重写缓存是硬要求**：否则 prompt sensitivity 会把成本线性放大。⁷
- **固定抽样 ID 列表**：同一批样本用于 baseline、prompt sensitivity、以及 deepseek vs gpt 对照；否则“差异来自样本”而不是“来自提示词/模型”。
- **记录重写参数**：temperature、top_p、max_tokens 等必须写入缓存文件头或 `manifest.json`，因为这些参数会改变输出随机性与编辑距离分布。⁸

Milestone 1：Baseline (Invariance) 三域跑通

范围：XSum / PubMedQA / WritingPrompts（与你开题一致）。

交付物与验收：

- `rewrite_{dataset}.jsonl`：至少包含 `id, label(human/ai), text, prompt_id, rewriter_model, rewritten_text`
- `metrics_{dataset}.json`：至少含 **F1、AUROC、TPR@FPR**（建议 FPR=1% 或 5% 两档）与混淆矩阵
- `baseline_invariance_table.csv`：三行数据集 × 多列指标
- 机制验收：
- human vs AI 的编辑距离特征在分布上可分（画直方图/箱线图）
- LR 能学到稳定边界（F1 明显高于随机）

论文侧对齐点（你在报告/论文里可以引用的“机制依据”）：

- RAIDAR 的关键观察是：在同一重写提示下，机器文本更少修改、人类文本更常被修改。 ⁹
- 其核心特征包含 bag-of-words edit 与 Levenshtein ratio（长度归一化）。 ¹⁰

Milestone 2: 提示词敏感性 (Prompt sensitivity)

做法：3-8 个提示词 × 3 个数据集，逐一跑 Invariance。

必须对齐/引用的论文结论：

- 论文 Figure 6 明确指出：不同提示词会显著影响最终检测性能，且没有单一提示词在所有数据源最优。 ¹¹

交付物：

- `prompt_pool.json`：提示词文本 + 类型标签 (fluency/style/expand/concise/...) + 是否长度控制 + 目标长度策略
- `prompt_sensitivity_{dataset}.csv`：prompt_id × 指标 (F1/AUROC/TPR@FPR)
- 一张总图：每个数据集上不同 prompt 的柱状图/折线图（用于写作“提示词敏感性”结果段）

长度控制（作为可配置项，与你开题一致）：

- 在 prompt 中显式加入约束，如“尽量保持与原文长度相近（±10%）/保持句子数相近/只做最小改动”。
- 同时保留“无长度控制”版本，形成 2 × prompt 的对照矩阵；否则你很难判断性能变化究竟来自“提示词语义”还是“长度变化”。 ¹²

Milestone 3: DeepSeek 替代 GPT-3.5 的严谨性对照

配置建议：

- 每数据集抽样 200（100 human + 100 AI），只跑 Invariance + LR。
- 两套重写：gpt-3.5-turbo（或固定快照）vs deepseek-chat。 ¹³

验收与写作要点（你可以直接写进论文方法/实验设置）：

- **分布一致性**：两种重写器下，编辑距离特征都呈现“human 改动更大、AI 改动更小”的同方向分离趋势。 ¹⁴
- **性能趋势一致性**：F1 不要求数值一致（模型不同是预期），但不应出现“换成 DeepSeek 后机制失效、接近随机”的反转。
- **结论段落模板**：
“我们以 GPT-3.5 作为参考重写器，对 deepseek-chat 的替代进行了小规模一致性验证；在三域上均观察到同方向的编辑距离分离趋势与可学习的分类边界，因此后续大规模实验使用 deepseek-chat 以控制成本。”

数据获取与“human/AI 配对”构造策略

你的 3 个主数据集是公开数据集，但 RAIDAR 类检测任务需要 **human 与 AI 两个来源标签**。因此你必须在工程里明确：

1) human 文本取自数据集原始人类文本；2) AI 文本用某个生成模型按规则生成，并与 human 在长度/体裁上尽量匹配。

XSum（新闻域）获取与配对

- 数据来源：XSum 官方仓库给出下载与预处理说明；同时也有常用数据集托管版本便于直接加载。¹⁵
- 建议的检测文本单位：如果你直接用 XSum 的一行 summary（极短），可能会让编辑距离特征方差过大；更稳妥的做法是从 document 中截取“段落级”片段（例如前 120–200 词），与 RAIDAR 的“段落级检测”设定更接近。¹⁶
- AI 配对生成（低成本版本）：用 deepseek-chat 生成“与 human 段落长度接近的新闻段落”，输入可用 summary 作为“标题/要点”，并加入长度约束。

PubMedQA（医学文本域）获取与配对

- 数据来源：PubMedQA 官网给出数据规模与下载入口；其 GitHub 仓库提供下载与切分脚本。¹⁷
- 关键落地建议：PubMedQA 字段较多（question、context/abstract、label 等），你应先用脚本打印字段统计，选择最适合“段落级检测”的 text 字段作为检测对象（通常是 abstract/context）。
- AI 配对生成：可用“题目/问题 + 关键句”生成一段解释性文本，或用标题生成摘要式段落，但务必加长度约束以免“长度差”成为捷径。

WritingPrompts（创意写作域）获取与配对

- 数据来源：WritingPrompts 数据集在 Hugging Face 上有标准数据卡与可直接加载版本。¹⁸
- human 文本：原始故事（story）。
- AI 配对生成：用同一 prompt 让生成模型写一段与 human 长度相近的故事段落（可控制字数/句子数），并记录生成模型/参数。

工程骨架与缓存规范（可直接照抄落地）

配置文件字段建议（YAML）

```
run_name: xsum_invariance_baseline
dataset:
  name: xsum
  source: huggingface
  split: train
  sample:
    n_human: 2000
    n_ai: 2000
    seed: 42
    id_list_path: assets/xsum_ids_seed42.json

ai_generation:
  model_provider: deepseek
  model: deepseek-chat
  prompt_template: "Write one news paragraph (~{target_words} words) based on this summary:
{summary}"
  length_control:
    target_words_from_human: true
  temperature: 0.7

rewriting:
  rewriter_provider: deepseek
  rewriter_model: deepseek-chat
```

```

temperature: 0
max_output_tokens: 512
prompt_pool_path: prompt_pool_v1.json
cache_dir: outputs/{run_id}/rewrite/

features:
  ngram_max_n: 4
  use_lev_ratio: true
  normalize_lower: true
  strip_punct: false

modeling:
  classifier: logistic_regression
  lr:
    C_grid: [0.1, 1.0, 10.0]
    class_weight: balanced
  split:
    test_size: 0.2
    seed: 42

evaluation:
  metrics: [f1, auroc, tpr_at_fpr_0_01, tpr_at_fpr_0_05]
  bootstrap_ci:
    n: 1000
    seed: 123

```

必须缓存的两类文件

- `ai_text.jsonl`：生成得到的 AI 文本（正类），否则每次实验都要重新生成。
- `rewrite.jsonl`：重写结果（按 `rewriter × prompt × text_id` 缓存），否则 `prompt sensitivity` 会重复烧钱。 7

重写、特征提取、训练评估的具体可执行步骤

重写 API (deepseek-chat) 示例

DeepSeek 文档明确其 API 与 OpenAI 兼容，并给出 `base_url` 写法与计费方式。 19

```

from openai import OpenAI
import os

client = OpenAI(
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url="https://api.deepseek.com"
)

def rewrite_once(text: str, prompt: str, temperature: float = 0.0) -> str:
    resp = client.chat.completions.create(
        model="deepseek-chat",
        messages=[{"role": "user", "content": f"{prompt}: {text}"}],

```

```

        temperature=temperature
    )
    return resp.choices[0].message.content.strip()

```

GPT-3.5 对照重写示例（小样本）

OpenAI 文档列出 gpt-3.5-turbo 快照与价格，并说明可用快照锁定版本以减少漂移。 ²⁰

```

from openai import OpenAI
import os

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

def rewrite_gpt35(text: str, prompt: str) -> str:
    resp = client.chat.completions.create(
        model="gpt-3.5-turbo-0125",
        messages=[{"role": "user", "content": f"{prompt}: {text}"},],
        temperature=0
    )
    return resp.choices[0].message.content.strip()

```

可复现性提醒：即便设 temperature=0，LLM 仍可能存在非确定性；OpenAI Cookbook 介绍了使用 `seed` 辅助提高可复现性的方法（需要同时固定其它参数）。 ²¹

特征提取伪代码（论文口径）

论文定义了：

- n-gram/词袋重合度（共有 n-gram 数/长度）； ²²
- Levenshtein ratio: $1 - \text{dist} / \max(\text{len})$ 。 ²³

```

from rapidfuzz.distance import Levenshtein

def ngrams(tokens, n):
    return [" ".join(tokens[i:i+n]) for i in range(len(tokens)-n+1)]

def bow_ngram_overlap(x, s, n_max=4):
    tx = x.split()
    ts = s.split()
    L = max(1, len(tx))
    feats = []
    for n in range(1, n_max+1):
        gx = set(ngrams(tx, n))
        gs = set(ngrams(ts, n))
        feats.append(len(gx & gs) / L)
    return feats

def lev_ratio(x, s):
    dist = Levenshtein.distance(x, s)

```

```

denom = max(1, max(len(x), len(s)))
return 1.0 - dist / denom

def featurize(x, s, n_max=4):
    return bow_ngram_overlap(x, s, n_max) + [lev_ratio(x, s)]

```

训练与调参流程（LR/XGBoost）

- **Logistic Regression（主力）**：对高维稀疏/少量连续特征都稳健；建议用 `class_weight=balanced`，并在 `C` 上做小网格搜索（0.1/1/10）。
- **XGBoost（备选/加分）**：当特征非线性或不同提示词特征交互明显时可能更好；但在你当前“成本优先、工程简化”的阶段，可先用 LR 做统一口径，XGBoost 作为对比补充。
- 评估切分：固定 `test_size=0.2` 与随机种子；在 prompt sensitivity 中，同一数据集也应固定切分以便比较。

评估指标、统计检验与实验设计细则

指标（你计划中的最小集合）

- **F1、Precision、Recall**：用于与你后续工作一致对比（RAIDAR 论文表格也以 F1 为主）。²³
- **AUROC**：用于观察阈值无关的可分性。
- **TPR@FPR**：建议报告 TPR@FPR=1% 与 5%（可贴近“低误报”场景）。

Prompt sensitivity 的实验设计要点

- prompt 池必须版本化（如 `prompt_pool_v1.json`），并写清每条 prompt 是否启用长度控制。
- 输出应包含：每个 prompt 的特征分布统计（均值/分位数）与指标表；以及“跨三域是否存在单一最优”。
- 论文已有结论可引用：prompt 会显著影响 F1，且无单一 prompt 最优。¹¹

DeepSeek vs GPT-3.5 对照的统计检验建议（轻量但严谨）

- **分布检验（描述性即可）**：画出 human/AI 的 `lev_ratio` 分布直方图与箱线图，并报告分位数差（如 P50、P75）。
- **性能差异置信区间**：对 test set 做 bootstrap（如 1000 次）计算 F1 的 95% CI；重点看“DeepSeek 是否仍明显高于随机”。
- **配对显著性（可选加分）**：对同一 test set 的两模型预测用 McNemar 检验（若你做最终论文写作，这一段很加分）。

资源、预算与风险控制

API 成本估算（把钱花在“对你创新有用”的地方）

DeepSeek 计费按 token 计，且区分 cache hit/miss；deepseek-chat 的公开价格在其文档中给出。²⁴
 OpenAI gpt-3.5-turbo 的价格在官方模型页面列出。²⁰

建议用一个简单公式在 `metrics.json` 旁边输出 `cost_estimate.json`：

- $cost \approx (T_{in}/1e6) * price_{in} + (T_{out}/1e6) * price_{out}$
- Prompt sensitivity 成本大头来自“重复次数 = 样本数 × prompt 数”，因此缓存与固定样本至关重要。

粗量级示例（仅作为预算量纲，不作为承诺）：

- 若三域各抽 2000 human + 2000 AI，共 12000 条；prompt 池 8 条 $\Rightarrow$ 96000 次重写调用。
- 若平均每次输入+输出各 300 tokens，则 $T_{in} \approx 28.8M$ 、 $T_{out} \approx 28.8M$ 。
- deepseek-chat（cache miss 计）约：输入 $28.8M \times \$0.27/1M$ + 输出 $28.8M \times \$1.10/1M \approx \39.5 。 25
- gpt-3.5-turbo 约：输入 $28.8M \times \$0.50/1M$ + 输出 $28.8M \times \$1.50/1M \approx \57.6 。 20

结论：在你设定的规模下，“prompt sensitivity”完全可控；真正会失控的是 **不缓存** 或者把 Equivariance/Uncertainty 拉进来导致调用倍增。

主要风险与缓解

- **风险：human/AI 配对不严谨（长度/内容差成捷径）**
缓解：强制长度控制策略；并报告 length 分布；必要时用“长度当作单独特征”的消融验证是否过拟合长度。 12
- **风险：提示词版本漂移（你后期改 prompt，旧结果不可比）**
缓解：prompt 池版本化；每次 run 输出 `prompt_pool_hash`。 26
- **风险：LLM 随机性导致结果波动**
缓解：固定 temperature；固定 seed（可用时）；至少重复 3 个随机种子做 train/test 切分，报告均值 ± 标准差。 27
- **风险：你用 DeepSeek 重写被质疑“偏离论文”**
缓解：Milestone 3 的小对照写进论文；并引用论文对“提示词敏感、无单一最优”的结论，强调你研究主线的合理性。 11

八周时间表（与开题主线无缝衔接）

Week 1-4 完成“可对齐基线 + 提示词敏感性 + DeepSeek 替代对照”；Week 5-8 进入你的创新点（集成与路由），同时保留少量预算做可选扩展。

周	任务	里程碑产物
Week 1	Milestone 0: 工程骨架、三域数据加载、human/AI 配对生成脚手架、缓存格式定稿	<code>configs/</code> 、可运行 <code>run.py</code> 、 <code>outputs/</code> 目录规范、首批 200 条 smoke test
Week 2	Milestone 1: 三域 Invariance baseline 全跑通；输出分布图与 baseline 总表	<code>baseline_invariance_table.csv</code> 、三域 <code>metrics.json</code> 、分布图
Week 3	Milestone 2: prompt 池 v1（3-8 条）+ 长度控制对照；三域 prompt sensitivity	<code>prompt_pool_v1.json</code> 、 <code>prompt_sensitivity_*.csv</code> 、柱状图/热力图
Week 4	Milestone 3: DeepSeek vs GPT-3.5 小样本一致性验证；形成可写入论文的方法段落	<code>rewrite_compare_*.csv</code> 、一致性结论段落草稿
Week 5	提示词集成（Ensemble）：K 个提示词各重写一次；分数层聚合（mean/weighted）	<code>ensemble_baseline.csv</code> 、与 best-single-prompt 对比
Week 6	路由（Routing）v1: 用 embedding/简单分类器选择 Top-1/Top-2 prompt 再重写	<code>routing_v1_metrics.csv</code> 、routing 代价/收益曲线
Week 7	消融与稳健性：长度控制、prompt 类型、样本规模敏感性；可选加做 1 个扩展域（Yelp/Code/ArXiv 任选）	消融表、扩展域小结果（可放附录）

周	任务	里程碑产物
Week 8	整理论文实验章节：实验设置、成本、复现细节、开源/复现说明；补齐图表与复现实验脚本	可提交的实验章节初稿 + 可复现实验包

补充说明（与你的创新点如何无缝衔接）：当你完成 Milestone 1-3，你将拥有“提示词敏感性数据事实（无单一最优）+ 可控成本的重写式检测基线 + DeepSeek 替代的严谨性证据”，随后做提示词集成（K 提示各重写一次再聚合）与提示词路由（先选提示再重写）就变成顺理成章的“工程增强”，并且能用你自己复现的数据直接回答“集成/路由是否同时提升效果并节省调用”的核心研究问题。 28

1 3 4 6 9 10 11 12 14 16 22 23 26 28 [arxiv.org](https://arxiv.org/pdf/2401.12970)

<https://arxiv.org/pdf/2401.12970>

2 19 <https://api-docs.deepseek.com/>

<https://api-docs.deepseek.com/>

5 13 20 [GPT-3.5 Turbo Model | OpenAI API](https://developers.openai.com/api/docs/models/gpt-3.5-turbo)

<https://developers.openai.com/api/docs/models/gpt-3.5-turbo>

7 24 25 https://api-docs.deepseek.com/quick_start/pricing-details-usd

https://api-docs.deepseek.com/quick_start/pricing-details-usd

8 <https://platform.openai.com/docs/api-reference/responses>

<https://platform.openai.com/docs/api-reference/responses>

15 [EdinburghNLP/XSum - Extreme Summarization](https://github.com/EdinburghNLP/XSum)

https://github.com/EdinburghNLP/XSum?utm_source=chatgpt.com

17 [PubMedQA Homepage](https://pubmedqa.github.io/)

https://pubmedqa.github.io/?utm_source=chatgpt.com

18 [euclaise/writingprompts • Datasets at Hugging Face](https://huggingface.co/datasets/euclaise/writingprompts)

https://huggingface.co/datasets/euclaise/writingprompts?utm_source=chatgpt.com

21 27 [https://developers.openai.com/cookbook/examples/](https://developers.openai.com/cookbook/examples/reproducible_outputs_with_the_seed_parameter/)

[reproducible_outputs_with_the_seed_parameter/](https://developers.openai.com/cookbook/examples/reproducible_outputs_with_the_seed_parameter/)

https://developers.openai.com/cookbook/examples/reproducible_outputs_with_the_seed_parameter/